GUÍA DE INTEGRACIÓN

One-Shot Biometric

Cómo usar la pasarela de verificación de identidad y firma electrónica de UniverseID desde el sistema de SMARTCIDI: usted configura su organización, nosotros ponemos la pantalla, la prueba de vida, el Registro Civil, la comparación facial y la firma. Paso a paso, con ejemplos en cURL, JavaScript y Python.

Versión 2.0 (modalidad pasarela)
Fecha 12 de octubre de 2026
Dirigido a SMARTCIDI — gerencia y equipo técnico
Elaborado por Dirección de Desarrollo — UniverseID / DARKCAM
Clasificación Confidencial — uso del cliente

Contenido

Contenido 2
1. ¿Qué es la modalidad pasarela? 3
2. Requisitos que debe cumplir SMARTCIDI 4
2.1 Configuración inicial (una sola vez) 4
2.2 Lo que debe tener su sistema 4
3. El recorrido en un vistazo 5
4. Antes de empezar: dirección y llave 5
5. Paso a paso con cURL 6
Paso 1 — Crear la sesión 6
Paso 2 — Enviar al cliente al enlace 7
Paso 3 — Obtener el resultado 9
Cancelar una sesión (opcional) 9
6. Webhooks: que nosotros le avisemos 10
Cómo comprobar que el aviso es nuestro 10
Receptor en JavaScript (Node.js) 10
Receptor en Python (Flask) 11
7. Ejemplos completos listos para ejecutar 11
JavaScript (Node.js 18 o superior, sin dependencias) 11
Python (requests) 12
8. Cuando algo sale mal 12
Fallos de la verificación 13
Reintentos desde su servidor 13
9. Cómo probar sin riesgo (sandbox) 13
10. Lista de salida a producción 14

Para numerar el indice: abrir en Word, clic derecho sobre esta tabla y elegir «Actualizar campos» (o pulsar F9).

+593 98 267 0640
info@gruporadical.com
www.gruporadical.com
Ponce Carrasco E8-06 y Av. Diego de Almagro, Edif. Almagro Plaza, piso 12, oficina 1213.

1. ¿Qué es la modalidad pasarela?

Piense en una **ventanilla de atención ya construida**. SMARTCIDI no tiene que levantar el mostrador, ni contratar a quien verifica documentos, ni conseguir la cámara: solo envía a su cliente a nuestra ventanilla y espera a que le avisemos cómo le fue. Eso es la pasarela de **One-Shot Biometric**.

En esta modalidad **UniverseID es el proveedor de la prueba de vida** (y de todo lo demás). SMARTCIDI únicamente:

1. **Configura su organización** en UniverseID (marca, documentos a firmar, dirección donde recibirá avisos).
2. **Crea una sesión** llamando a nuestra API con los datos de la persona.
3. **Envía a esa persona a un enlace** (`verification_url`) donde hace su verificación.
4. **Recibe el resultado**: un aviso automático (webhook) con los PDF firmados, y una consulta de estado por API.

Etapa	Quién la hace	Dónde ocurre
Pantalla del cliente final, con su marca	UniverseID	Página alojada por nosotros
Prueba de vida (cámara)	UniverseID	Página alojada por nosotros
Validación en el Registro Civil	UniverseID	Servidor de UniverseID
Comparación del rostro con la foto de la cédula	UniverseID	Servidor de UniverseID
Emisión del certificado de un solo uso (24 h)	UniverseID	Servidor de UniverseID
Firma de los documentos de SMARTCIDI	UniverseID	Servidor de UniverseID
Crear la sesión, enviar al cliente al enlace, recibir el resultado	**SMARTCIDI**	Su sistema (3 llamadas simples)

> **Lo que SMARTCIDI NO necesita construir**
> No necesita pantalla de cámara, ni el SDK de AWS, ni su propio motor de prueba de vida, ni manejar certificados o llaves privadas. Nada de eso pasa por su sistema.

2. Requisitos que debe cumplir SMARTCIDI

Esta es la lista de verificación. Se divide en lo que hacemos juntos al inicio (configuración) y lo que debe tener su sistema.

2.1 Configuración inicial (una sola vez)

#	Qué	Detalle	Quién
1	Alta de la organización	Nombre, RUC, correo de contacto y límites de uso (diario y mensual)	UniverseID, con sus datos
2	Dirección de webhooks	Una URL `https://` pública de SMARTCIDI. **Debe indicarse en el alta**: de ahí se genera el secreto de firma (`whsec_…`)	SMARTCIDI entrega la URL; UniverseID la registra
3	API key de pruebas `dk_test_…`	Se entrega una sola vez. Guárdela como secreto. La de producción (`dk_live_…`) se emite al pasar a producción	UniverseID / panel de la organización
4	Documentos a firmar (plantillas)	Cada plantilla es un PDF más la posición del sello de firma. Se cargan en el panel de la organización y cada una recibe un `document_id` (UUID)	SMARTCIDI en el panel
5	Marca (opcional)	Logo, colores y textos de la página que verá su cliente	SMARTCIDI en el panel

2.2 Lo que debe tener su sistema

#	Requisito	Detalle
1	Servidor que llame a nuestra API	Las llamadas salen **siempre desde su servidor** con la API key; nunca desde el navegador ni desde una app móvil
2	Receptor de webhooks en HTTPS	Dirección pública con certificado válido (no `localhost` ni IP privada: la bloqueamos por seguridad). En desarrollo, use un túnel (ngrok o Cloudflare Tunnel)
3	Datos del titular	Cédula (10 dígitos), código dactilar (6 o más caracteres), nombres, apellidos y correo electrónico
4	Dos direcciones de retorno	`return_url` (obligatoria: adónde regresa el cliente al terminar) y `cancel_url` (opcional: si cancela)
5	Informar al titular	Antes de enviarlo al enlace, informe que se tratarán sus datos biométricos. Nuestra página también lo indica antes de comenzar
6	Poder reconstruir el estado	Guarde el `session_id` de cada operación junto a su propio número de pedido; permite conciliar y pedir soporte
7	Guardar los PDF firmados	Los avisos traen la dirección de descarga de cada PDF; descárguelo y guárdelo en sus sistemas

> **Importante**
> La API key es como la llave de su casa: no la escriba en páginas web, apps ni repositorios. Si sospecha que se filtró, avísenos y la revocamos de inmediato.

3. El recorrido en un vistazo

Paso	Quién	Qué ocurre
1	SMARTCIDI → API	`POST /sessions` con los datos de la persona y los documentos a firmar. Respuesta: `verification_url`
2	SMARTCIDI → su cliente	Lo envía a la `verification_url` (redirección, enlace por correo o WhatsApp, o código QR)
3	Su cliente → UniverseID	Hace la prueba de vida en nuestra página. El resto (Registro Civil, rostro, certificado) corre solo
4	UniverseID → SMARTCIDI	Aviso `documents.signed` (webhook) con los PDF firmados; el cliente regresa a su `return_url`
4b	SMARTCIDI → API	Respaldo: `GET /sessions/{id}` muestra el estado en cualquier momento

La sesión dura **10 minutos** desde que se crea. Si el cliente no termina a tiempo, pasa a `expired` y debe crear una sesión nueva.

4. Antes de empezar: dirección y llave

Ambiente	Dirección base	Llave
Pruebas (sandbox)	En la demo local: `http://localhost:8080/api/one-shot-biometric`. Para su integración, UniverseID le indicará la de pruebas	`dk_test_…`
Producción	`https://api.universe-id.com/api/one-shot-biometric`	`dk_live_…`

El ambiente lo decide el prefijo de la llave. En pruebas no hay cobro, el Registro Civil y la biometría son simulados, y la página del cliente muestra botones para simular éxito o fallo.

Toda llamada lleva el encabezado `Authorization: Bearer <su llave>`:

```
export DK_BASE="http://localhost:8080/api/one-shot-biometric"
export DK_API_KEY="dk_test_xxxxxxxxxxxxxxx"
export DK_DOC_ID="a3896ace-f382-4658-9746-5eb956c04023"     # id de su plantilla de documento

# Comprobación: debe responder success:true con sus cupos de uso
curl -sS "$DK_BASE/usage" -H "Authorization: Bearer $DK_API_KEY" | jq .
```

5. Paso a paso con cURL

Todos estos comandos fueron ejecutados de principio a fin el 12 de octubre de 2026.

Paso 1 — Crear la sesión

Es como sacar un turno en la ventanilla: la sesión guarda a quién vamos a verificar y qué documentos firmará.

```
curl -sS -X POST "$DK_BASE/sessions" \
  -H "Authorization: Bearer $DK_API_KEY" -H "Content-Type: application/json" \
  -d '{
    "cedula": "1712345678",
    "codigo_dactilar": "V4444V4444",
    "nombres": "JUAN CARLOS",
    "apellidos": "PEREZ GARCIA",
    "email": "juan.perez@smartcidi.com",
    "return_url": "https://smartcidi.com/gracias",
    "cancel_url": "https://smartcidi.com/cancelado",
    "document_ids": ["'"$DK_DOC_ID"'"]
  }'

# Respuesta
{ "success": true, "message": "Sesión creada exitosamente",
  "data": {
    "session_id": "40dccfd9-d69b-42f2-a047-5365fb62a5c8",
    "verification_url": "https://app.universe-id.com/verify/40dccfd9-…",
    "expires_at": "2026-10-12T15:41:44.116Z",
    "environment": "sandbox",
    "process_steps": ["liveness", "civil_registry", "face_comparison", "certificate_issuance"] },
  "request_id": "req_1791819104007_f21f94fb4556cf69" }
```

Campo	¿Obligatorio?	Notas
`cedula`	Sí	10 dígitos, solo números
`codigo_dactilar`	Sí	6 o más caracteres
`nombres`, `apellidos`	Sí	Mínimo 2 caracteres cada uno; se muestran al cliente en la pantalla de inicio
`email`	Sí	Correo válido de la persona
`return_url`	Sí	Adónde regresa el cliente al terminar
`cancel_url`	No	Adónde regresa si cancela
`provincia`, `ciudad`	No	Si no se envían, se asumen `PICHINCHA` y `QUITO`
`document_ids`	Sí	Uno o más UUID de plantillas **activas de su organización**; si alguno no existe responde 400

Guarde el `session_id`: lo usará para consultar el estado y para conciliar con los avisos.

Paso 2 — Enviar al cliente al enlace

Entregue la `verification_url` a su cliente por el medio que prefiera: redirección del navegador, enlace en un correo o WhatsApp, o un código QR. No necesita hacer nada más: a partir de aquí **trabajamos nosotros**. Esto es lo que ve su cliente:

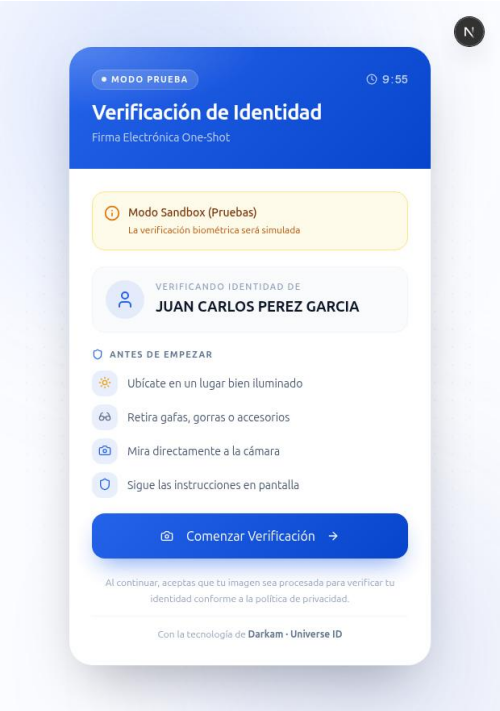


Figura 1. *Pantalla de inicio: muestra el nombre de la persona, el tiempo restante y las recomendaciones.*

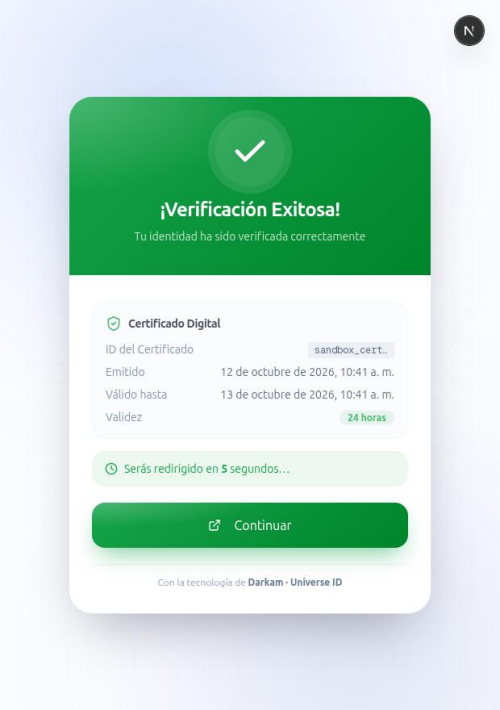


Figura 2. *Pantalla final de éxito; luego se le redirige a su return_url.*

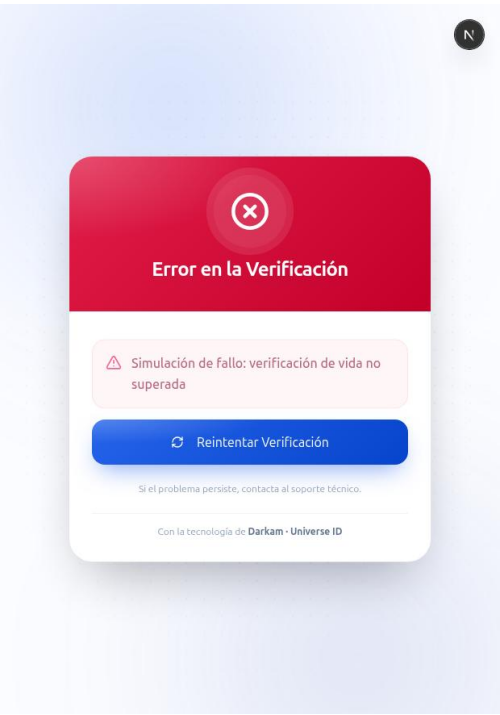


Figura 3. *Si la verificación falla (aquí, simulada), el cliente ve el error y puede volver a su sitio.*

> **En pruebas**
> En sandbox la cámara se reemplaza por dos botones: «Simular Verificación Exitosa» y «Simular Verificación Fallida». En producción se abre la cámara y se hace la prueba de vida real.

Paso 3 — Obtener el resultado

La vía principal es el **webhook** (sección 6). Como respaldo, y para conciliar, consulte la sesión:

```
curl -sS "$DK_BASE/sessions/$SID" -H "Authorization: Bearer $DK_API_KEY"

# Respuesta con la sesión terminada
{ "success": true,
  "data": {
    "session_id": "40dccfd9-…", "status": "completed", "environment": "sandbox",
    "created_at": "2026-10-12T15:31:44Z", "completed_at": "2026-10-12T15:33:45Z",
    "citizen": { "cedula_masked": "****345678", "nombres": "…", "apellidos": "…", "email_masked": "j***@smartcidi.com"
},
    "liveness":        { "completed": true, "confidence": 95.5, "status": "SUCCEEDED" },
    "face_comparison": { "completed": true, "similarity_score": 92.3, "is_match": true },
    "error": null } }
```

El `status` va avanzando solo; usted únicamente debe distinguir entre «en progreso» y «terminal»:

Tipo	Estados	Qué hacer
En progreso	`pending`, `liveness_started`, `liveness_completed`, `civil_registry_validating`, `civil_registry_completed`, `face_comparison_validating`, `face_comparison_completed`, `certificate_issuing`	Esperar y volver a consultar cada 3 segundos
Terminal exitoso	`completed`	La persona quedó verificada y los documentos se firman; los PDF llegan por webhook
Terminal con fallo	`liveness_failed`, `civil_registry_failed`, `age_validation_failed`, `face_comparison_failed`, `failed`	Ver `data.error.code` y `data.error.message`; crear una sesión nueva si se desea reintentar
Terminal sin resultado	`cancelled`, `expired`	Crear una sesión nueva

> **No espere material de firma en esta consulta**
> Por seguridad, el certificado y su llave privada **nunca** se entregan a SMARTCIDI: se usan en nuestros servidores para firmar y luego se descartan. Lo que usted recibe son los PDF ya firmados (sección 6).

Cancelar una sesión (opcional)

```
curl -sS -X DELETE "$DK_BASE/sessions/$SID" -H "Authorization: Bearer $DK_API_KEY" \
  -H "Content-Type: application/json" -d '{"reason":"El cliente pidió cancelar"}'
```

No tiene efecto si la sesión ya está `completed`, `failed` o `expired`.

6. Webhooks: que nosotros le avisemos

En lugar de preguntar cada pocos segundos, deje que UniverseID le llame, como el timbre de una puerta. Cuando los documentos quedan firmados, enviamos un `POST` a la dirección de webhooks de su organización con el evento `documents.signed`:

```
{
  "event": "documents.signed",
  "session_id": "40dccfd9-d69b-42f2-a047-5365fb62a5c8",
  "organization_id": "fa36cebd-555c-4f19-b01f-2fede7437b2b",
  "environment": "sandbox",
  "timestamp": "2026-10-12T15:32:50.000Z",
  "signer": { "cedula_masked": "****345678", "nombres": "…", "apellidos": "…" },
  "certificate": { "certificate_id": "…", "issued_at": "…", "expires_at": "…" },
  "documents": [
    { "document_id": "a3896ace-…", "name": "Contrato de servicios SMARTCIDI",
      "status": "signed", "signed_url": "https://…/signed-documents-public/….pdf" }
  ],
  "signature": "6f1c…"
}
```

Cada documento trae `status` (`signed` o `failed`) y, si fue firmado, su `signed_url` para descargarlo. Si uno falla (`failed`), trae `error_code` y `error_message`; el certificado ya se emitió, así que **no cree una sesión nueva**: el documento se reintenta desde el panel sin costo adicional.

Cómo comprobar que el aviso es nuestro

Cada aviso lleva el campo `signature` y los encabezados `X-Webhook-Signature` y `X-Webhook-Event`. La firma es el **SHA-256** (hexadecimal) del texto JSON del aviso **sin el campo `signature`**, seguido de su secreto `whsec_…` (sin separador). **Siempre verifíquela** antes de confiar en el contenido.

Receptor en JavaScript (Node.js)

```
import { createServer } from "node:http";
import { createHash, timingSafeEqual } from "node:crypto";
const SECRET = process.env.DK_WEBHOOK_SECRET;                  // whsec_...

createServer((req, res) => {
  const partes = [];
  req.on("data", (c) => partes.push(c));
  req.on("end", () => {
    const { signature, ...aviso } = JSON.parse(Buffer.concat(partes).toString("utf8"));
    const esperada = createHash("sha256").update(JSON.stringify(aviso) + SECRET).digest("hex");
    const ok = typeof signature === "string" && signature.length === esperada.length &&
               timingSafeEqual(Buffer.from(signature), Buffer.from(esperada));
    if (!ok) { res.writeHead(401).end(); return; }

    for (const d of aviso.documents ?? []) console.log(d.name, d.status, d.signed_url);  // descargue y guarde el PDF
    res.writeHead(200).end("ok");                                  // responda 2xx rápido
  });
}).listen(9099);
```

Receptor en Python (Flask)

```
import hashlib, hmac, json, os
from flask import Flask, request, abort
app = Flask(__name__)
SECRET = os.environ["DK_WEBHOOK_SECRET"]

@app.post("/webhooks/universe-id")
def webhook():
    aviso = json.loads(request.get_data(), object_pairs_hook=dict)   # conserva el orden de los campos
    firma = aviso.pop("signature", "")
    esperada = hashlib.sha256((json.dumps(aviso, separators=(",", ":"), ensure_ascii=False) +
SECRET).encode()).hexdigest()
    if not hmac.compare_digest(firma, esperada):
        abort(401)
    for d in aviso.get("documents", []):
        print(d["name"], d["status"], d.get("signed_url"))           # descargue y guarde el PDF
    return "ok"
```

- Responda con `2xx` en pocos segundos. Si no lo hace, reintentamos hasta 3 veces.
- Puede llegar el mismo aviso más de una vez: use el `session_id` para no procesarlo dos veces.
- Las direcciones `signed_url` son de descarga directa: descargue el PDF apenas llegue el aviso y guárdelo en sus sistemas.
- **Los fallos de verificación** (rostro que no coincide, cédula inexistente, etc.) se consultan con `GET /sessions/{id}` y se reflejan en el regreso del cliente a su `return_url`. Su flujo debe contemplarlos (sección 8).

> **Para desarrolladores**
> Su dirección de webhooks debe ser pública y `https://`. En su computador de desarrollo use un túnel (por ejemplo `ngrok http 9099`) y entregue esa dirección en la configuración de la organización.

7. Ejemplos completos listos para ejecutar

Ambos programas crean la sesión, muestran el enlace para el cliente y esperan el resultado. Defina `DK_BASE`, `DK_API_KEY` y `DK_DOC_ID` en el entorno.

JavaScript (Node.js 18 o superior, sin dependencias)

```
const BASE = process.env.DK_BASE, KEY = process.env.DK_API_KEY;
const sleep = (ms) => new Promise((r) => setTimeout(r, ms));

async function api(metodo, ruta, json) {
  const r = await fetch(BASE + ruta, {
    method: metodo,
    headers: { Authorization: `Bearer ${KEY}`, "Content-Type": "application/json" },
    body: json ? JSON.stringify(json) : undefined,
  });
  const p = await r.json();
  if (!r.ok) throw new Error(`${ruta} → ${r.status} ${p.message} (request_id ${p.request_id})`);
  return p.data;
}

// 1 · Crear la sesión
const s = await api("POST", "/sessions", {
```

```
  cedula: "1712345678", codigo_dactilar: "V4444V4444",
  nombres: "JUAN CARLOS", apellidos: "PEREZ GARCIA", email: "juan.perez@smartcidi.com",
  return_url: "https://smartcidi.com/gracias", cancel_url: "https://smartcidi.com/cancelado",
  document_ids: [process.env.DK_DOC_ID],
});
console.log("2 · Envíe al cliente a:", s.verification_url);

// 3 · Esperar el resultado (el webhook es la vía principal; esto es el respaldo)
const FINALES = ["completed", "liveness_failed", "civil_registry_failed", "age_validation_failed",
                 "face_comparison_failed", "failed", "cancelled", "expired"];
let r;
do { await sleep(3000); r = await api("GET", `/sessions/${s.session_id}`); console.log("   estado:", r.status); }
while (!FINALES.includes(r.status));
console.log(r.status === "completed" ? "☑ Verificación exitosa" : `✗ Terminó en ${r.status}`, r.error ?? "");
```

Python (requests)

```
import os, sys, time, requests
BASE, KEY = os.environ["DK_BASE"], os.environ["DK_API_KEY"]
H = {"Authorization": f"Bearer {KEY}"}

def api(metodo, ruta, **kw):
    r = requests.request(metodo, BASE + ruta, headers=H, timeout=30, **kw)
    p = r.json()
    if r.status_code >= 400:
        sys.exit(f"{ruta} → {r.status_code} {p.get('message')} (request_id {p.get('request_id')})")
    return p["data"]

# 1 · Crear la sesión
s = api("POST", "/sessions", json={
    "cedula": "1712345678", "codigo_dactilar": "V4444V4444",
    "nombres": "JUAN CARLOS", "apellidos": "PEREZ GARCIA", "email": "juan.perez@smartcidi.com",
    "return_url": "https://smartcidi.com/gracias", "cancel_url": "https://smartcidi.com/cancelado",
    "document_ids": [os.environ["DK_DOC_ID"]]})
print("2 · Envíe al cliente a:", s["verification_url"])

# 3 · Esperar el resultado (el webhook es la vía principal; esto es el respaldo)
FINALES = {"completed", "liveness_failed", "civil_registry_failed", "age_validation_failed",
           "face_comparison_failed", "failed", "cancelled", "expired"}
while True:
    time.sleep(3)
    r = api("GET", f"/sessions/{s['session_id']}")
    print("   estado:", r["status"])
    if r["status"] in FINALES:
        break
print("☑ Verificación exitosa" if r["status"] == "completed" else f"✗ Terminó en {r['status']}", r.get("error") or "")
```

8. Cuando algo sale mal

Los errores de la API llegan con `success: false`, un `message` en español y, casi siempre, un `request_id`. Guarde el `request_id`: es lo que necesitamos para ayudarle.

```
{ "success": false, "message": "Uno o más documentos no existen, están inactivos o no pertenecen a la organización",
  "error": "documents_not_found", "request_id": "req_1791819292913_0ac0ea25bc119e39" }      // HTTP 400

{ "success": false, "message": "Datos de entrada inválidos",
  "errors": { "cedula": ["La cédula debe tener exactamente 10 dígitos"] } }                  // HTTP 422
```

+593 98 267 0640
info@gruporadical.com
www.gruporadical.com
Ponce Carrasco E8-06 y Av. Diego de Almagro, Edif. Almagro Plaza, piso 12, oficina 1213.

HTTP	Qué significa	Qué hacer
400	Algún `document_id` no existe, está inactivo o no es de su organización	Revise el id en el panel
401	API key inválida, revocada o ausente	Revise `Authorization: Bearer …`
403	Organización inactiva	Contacte a UniverseID
404	La sesión no existe o es de otra organización	Revise el `session_id`
410	La sesión expiró	Cree una sesión nueva
422	Datos mal formados (el detalle viene en `errors`, campo por campo)	Corrija y reenvíe
429	Superó su límite diario o mensual	Espere o solicite ampliar el cupo
5xx	Falla temporal de nuestro lado	Reintente con espera creciente (2, 4, 8 s)

Fallos de la verificación

Si la persona no pasa la verificación, la sesión termina en un estado de fallo y `data.error` explica por qué:

Estado	Causa típica
`liveness_failed`	La prueba de vida no se aprobó (por ejemplo, mala iluminación o rostro no visible)
`civil_registry_failed`	El Registro Civil no encontró la cédula o los datos no coinciden
`age_validation_failed`	La persona es menor de edad
`face_comparison_failed`	El rostro no coincide con la foto de la cédula (`FACE_MISMATCH`)
`failed`	Error genérico de infraestructura

En todos los casos la sesión está terminada: para reintentar, **cree una sesión nueva** y envíe al cliente a su nuevo enlace.

Reintentos desde su servidor

- Errores **4xx**: son del pedido; corrija antes de reintentar.
- Errores **5xx** o cortes de red: reintente con espera creciente. Antes de crear otra sesión, revise que no se haya creado la anterior (guarde su `session_id`).

9. Cómo probar sin riesgo (sandbox)

Aspecto	En sandbox (dk_test_)	En producción (dk_live_)
Cobro	Ninguno	Según el acuerdo comercial
Prueba de vida	Simulada: botones «Exitosa» / «Fallida», sin cámara	Cámara y prueba de vida real
Registro Civil	Simulado (devuelve una persona de prueba)	Consulta real

Aspecto	En sandbox (dk_test_)	En producción (dk_live_)
Certificado	De prueba, sin validez legal	Emitido por la entidad certificadora
Documentos firmados	Firma real e intacta, pero el lector de PDF mostrará «identidad del firmante desconocida»: el certificado de prueba no pertenece a una entidad reconocida	Firma con validez legal, verificable en los validadores oficiales

Escenarios para probar su manejo de errores: cree una sesión, abra el enlace y pulse **«Simular Verificación Fallida»**. La sesión termina en un estado de fallo y puede revisar cómo reacciona su sistema.

> **Los datos de la persona en pruebas**
> En sandbox el Registro Civil es simulado, por lo que el nombre devuelto no será el que usted envió. Es normal y no ocurre en producción.

10. Lista de salida a producción

- ☐ Recorrido completo exitoso en sandbox, y también el caso de fallo («Simular Verificación Fallida»).
- ☐ Receptor de webhooks en `https://` verificando la firma, y sin procesar dos veces la misma sesión.
- ☐ Los PDF firmados se descargan al recibir el aviso y se guardan en sus sistemas.
- ☐ La llave `dk_live_…` guardada en un gestor de secretos (nunca en el código ni en el navegador).
- ☐ Plantillas de documento cargadas y probadas en el ambiente de producción.
- ☐ Marca (logo y colores) revisada en la página del cliente final.
- ☐ Registro (log) del `session_id` y del `request_id` de cada operación.
- ☐ Texto informativo al titular sobre el tratamiento de sus datos biométricos aprobado por su área legal.
- ☐ Acuerdo comercial y tarifas firmados (se entregan en un anexo aparte).

> **¿Necesita ayuda?**
> Envíe a su contacto de UniverseID el `request_id` y el `session_id` de la operación que falló. Con esos dos datos podemos ver exactamente qué ocurrió.